

UNIFIED OCR PLATFORM

Full System Architecture & Technical Specification

Version 1.0 · June 2025

Confidential — Internal Use Only

1. Executive Summary

The Unified OCR Platform is an AI-powered SaaS product that intelligently matches users to the OCR solution tier that best fits their specific document processing needs. Rather than exposing the underlying model names, the platform presents a guided experience: a conversational AI agent interviews the user about their use case, retrieves relevant technical knowledge via Retrieval-Augmented Generation (RAG), and recommends the appropriate subscription tier. Users can then validate the recommendation through a live document demo, and complete a purchase — all within a single seamless interface.

Core Value Proposition

- Users do not need to understand OCR technology — the agent does the work.
- Internal OCR model identities remain hidden; only tier names are exposed publicly.
- The platform drives higher conversion by matching users to the right product before purchase.
- A live demo experience eliminates buyer hesitation and reduces churn post-purchase.

2. System Overview

2.1 High-Level Architecture

The platform is composed of five major layers that communicate via well-defined APIs:

Layer	Component	Responsibility
Presentation	Next.js Frontend (React)	Landing page, chatbot UI, demo widget, checkout
Gateway	Nginx + Django REST API	Auth, routing, rate-limiting, session management
Intelligence	LLM Agent + RAG Engine	Conversation management, knowledge retrieval, tier recommendation
Processing	OCR Worker Service	HuggingFace model inference, queue management
Data	PostgreSQL, Pinecone, Redis, S3	Persistence, vector search, caching, file storage

2.2 User Journey (End-to-End)

The following sequence describes the complete flow from anonymous visitor to paying subscriber:

User Journey Flow

- Step 1 — Landing Page: Visitor arrives at the marketing site. Hero section showcases use cases and a prominent 'Try the AI Advisor' CTA.
- Step 2 — AI Advisor Chat: User opens the chatbot. The agent asks structured discovery questions (document type, language, complexity, volume, integration needs).
- Step 3 — RAG Lookup: Agent embeds user responses and retrieves matching chunks from the knowledge base (research papers + tier spec sheets).
- Step 4 — Recommendation: Agent surfaces 1–2 recommended tier(s) with a plain-language explanation of why they match the user's needs.
- Step 5 — Live Demo: User uploads a sample document. The recommended tier's OCR pipeline processes it live and returns extracted text/structured data.
- Step 6 — Checkout: User registers, selects tier, and completes Stripe-powered subscription purchase.
- Step 7 — Dashboard: Authenticated user accesses their OCR API key, usage stats, and upload history.

3. Frontend Architecture

3.1 Technology Stack

Technology	Purpose / Justification
Next.js 14 (App Router)	React framework with SSR/SSG; SEO-optimised landing pages + client interactivity
TypeScript	Type safety across all components and API calls
Tailwind CSS	Utility-first styling; rapid UI development
shadcn/ui	Accessible, composable component library built on Radix UI
Framer Motion	Smooth animations for landing page and chat transitions
React Query (TanStack)	Server-state management, caching, background refetching
Zustand	Lightweight client-state management for chat session and demo state
Stripe.js	PCI-compliant payment element embedded in the checkout flow
Vercel / AWS Amplify	Hosting with edge CDN for global performance

3.2 Page & Component Structure

The application is structured into four primary route groups:

- / — Public Marketing Site
 - Hero section with animated tagline and 'Try AI Advisor' button
 - Use-case showcase cards (invoice processing, scientific PDFs, handwritten forms, etc.)
 - Pricing section — shows tier names and feature matrix, NOT model names
 - Social proof / testimonials section
 - FAQ and footer
- /advisor — AI Chatbot + Demo Widget
 - Full-screen split layout: conversation panel (left) + document preview panel (right)
 - Chat messages rendered with streaming via Server-Sent Events (SSE)
 - Recommendation card component — appears after agent concludes discovery
 - Demo upload zone — drag-and-drop, supports PDF/PNG/JPG up to 10MB
 - OCR result viewer — side-by-side original doc and extracted text/JSON
- /checkout — Registration & Payment
 - Step 1: Email + password registration (or OAuth via Google)
 - Step 2: Tier confirmation with feature summary
 - Step 3: Stripe payment element (card, Google Pay, Apple Pay)

- Step 4: Success screen with API key reveal and onboarding CTA
- /dashboard — Authenticated User Area
 - API key management (view, rotate, revoke)
 - Usage meters: requests used / tier limit, reset date
 - Upload history with re-downloadable results
 - Account settings and subscription management (upgrade/cancel via Stripe portal)

3.3 Chat UI Implementation

The chatbot is the centrepiece of the product. Key implementation details:

- Streaming responses: The Django backend proxies LLM tokens via SSE. The frontend uses the EventSource API to render each token as it arrives, giving a typing effect.
- Conversation state: Chat history is stored in Zustand and persisted to localStorage for session continuity. On authentication it is synced to the backend.
- Typing indicators: While the agent is processing, an animated 'thinking' indicator is shown. The indicator hides the backend latency from RAG retrieval.
- Recommendation card: When the agent produces a recommendation, a structured JSON payload is embedded in the response. The frontend detects this and renders a styled card rather than raw text.
- Demo integration: The recommendation card includes an 'Upload & Test' button that activates the right panel. The demo pipeline automatically uses the recommended tier without requiring user configuration.

4. Backend Architecture (Django)

4.1 Django Project Structure

The backend is a Django 4.2 project using Django REST Framework (DRF). It is structured as a monorepo with clearly separated Django apps:

Project Layout

ocr_platform/	
├── core/	← Project settings, WSGI/ASGI entry, URL root
├── accounts/	← User model, JWT auth, OAuth, subscription status
├── advisor/	← Chat session model, LLM proxy, SSE streaming endpoint
├── rag/	← Document ingestion pipeline, Pinecone client, retrieval logic
├── ocr_engine/	← OCR job model, Celery task definitions, HuggingFace model loader
├── billing/	← Stripe integration, webhook handler, usage metering
├── demo/	← Unauthenticated demo endpoint (rate-limited), temp file handling
└── api/	← Versioned API router (v1), serializers, permissions

4.2 Tech Stack (Backend)

Technology	Purpose
Django 4.2 + DRF	Core web framework and REST API layer
djangorestframework-simplejwt	JWT-based access/refresh token authentication
django-allauth	Social OAuth (Google, GitHub) sign-in
Celery 5	Async task queue for OCR processing jobs
Redis 7	Celery broker, result backend, rate-limit counters, chat session cache
PostgreSQL 15	Primary relational database (users, subscriptions, jobs, chat sessions)
django-storages + boto3	S3-compatible storage for uploaded files and OCR results
Pinecone SDK	Vector database client for RAG retrieval
LangChain	LLM orchestration: prompt templates, RAG chain, conversation memory
openai / anthropic SDK	LLM API calls (Claude or GPT-4o; swappable via config)
Stripe SDK	Subscription lifecycle, payment intents, webhook handling
Gunicorn + Uvicorn	WSGI (API) + ASGI (SSE/WebSocket) concurrent servers
dj-database-url	12-factor DB config from environment variables

django-cors-headers	CORS policy for frontend domain
Sentry SDK	Error tracking and performance monitoring

4.3 API Endpoints

All endpoints are prefixed `/api/v1/`. Authentication via Bearer JWT token unless marked PUBLIC.

Method + Path	Auth	Description
POST <code>/auth/register/</code>	PUBLIC	Create account with email + password
POST <code>/auth/login/</code>	PUBLIC	Obtain JWT access + refresh tokens
POST <code>/auth/refresh/</code>	PUBLIC	Rotate access token using refresh token
GET <code>/auth/me/</code>	JWT	Fetch current user profile and subscription tier
POST <code>/advisor/session/</code>	PUBLIC (rate-limited)	Start a new chat session — returns <code>session_id</code>
POST <code>/advisor/message/</code>	PUBLIC (rate-limited)	Send user message; returns SSE stream of agent reply
GET <code>/advisor/session/{id}/history/</code>	JWT	Retrieve full conversation history
POST <code>/demo/upload/</code>	PUBLIC (rate-limited)	Upload file for live OCR demo; returns <code>job_id</code>
GET <code>/demo/result/{job_id}/</code>	PUBLIC	Poll or stream OCR demo result
POST <code>/billing/checkout/</code>	JWT	Create Stripe checkout session for selected tier
POST <code>/billing/webhook/</code>	PUBLIC (Stripe-sig)	Receive Stripe webhook events
GET <code>/billing/portal/</code>	JWT	Get Stripe customer portal URL for self-service
GET <code>/ocr/jobs/</code>	JWT	List user's OCR job history
POST <code>/ocr/jobs/</code>	JWT	Submit a new OCR job (authenticated, counted against quota)
GET <code>/ocr/jobs/{id}/</code>	JWT	Fetch job status and result
GET <code>/dashboard/usage/</code>	JWT	Current period usage vs tier limit

4.4 Django Apps — Detailed

4.4.1 accounts app

- Custom User model extending `AbstractBaseUser` — email as primary identifier (no username).
- `SubscriptionProfile` model: one-to-one with `User`, stores Stripe customer ID, current tier, quota limits, and reset date.
- JWT middleware validates every request and attaches the user + subscription tier to request context.
- Throttle classes: `AnonymousRateThrottle` (10/min for demo + advisor) and `UserRateThrottle` (per-tier limits).

4.4.2 advisor app

- `ChatSession` model: `session_id` (UUID), `user` (nullable FK), `created_at`, `last_activity`.
- `ChatMessage` model: `session` FK, `role` (user/assistant), `content` (text), `metadata` (JSON — stores recommendation payload when present).
- `POST /advisor/message/` creates a Celery task for LLM inference. The endpoint returns an SSE response using Django's `StreamingHttpResponse` with an async generator that yields tokens from the LLM stream.
- The advisor app imports the RAG engine (from rag app) to augment the LLM context with retrieved knowledge chunks before generating each response.

4.4.3 rag app

The RAG system is responsible for the agent's knowledge. It has two sub-systems:

Ingestion Pipeline (offline, admin-triggered):

- Admin uploads research papers (PDF) and tier spec sheets (Markdown/PDF) via Django Admin.
- A Celery task runs document parsing using LangChain's `PyPDFLoader` + `MarkdownTextSplitter`.
- Each chunk is embedded using a sentence-transformer model (e.g. BAAI/bge-large-en-v1.5, also hosted on HuggingFace Inference).
- Embeddings + metadata (`source`, `doc_type`: 'research_paper' | 'tier_spec', `tier_id`) are upserted to Pinecone.

Retrieval Pipeline (online, per request):

- User message is embedded using the same sentence-transformer.
- Pinecone similarity search returns top-K chunks (K=6), filtered by metadata where appropriate.
- Retrieved chunks are assembled into a context block injected into the LLM system prompt via a LangChain RAG chain.
- LLM is instructed to reason over both research paper evidence and tier spec capabilities to justify its recommendation.

4.4.4 ocr_engine app

- OCRJob model: job_id (UUID), user FK, tier used, input_file_path (S3 key), status (queued/processing/done/failed), result_path (S3 key), created_at, completed_at.
- Celery task process_ocr_job: downloads file from S3, loads the appropriate HuggingFace model pipeline, runs inference, uploads result JSON to S3, updates job status.
- Model selection is driven by tier_id — a mapping table in settings.py maps tier slugs to HuggingFace model IDs and inference endpoint URLs.
- For demo jobs (unauthenticated), a lightweight fast model is always used regardless of the recommended tier, to minimise latency and cost.

4.4.5 billing app

- On checkout initiation, create or retrieve Stripe Customer linked to the User.
- Create a Stripe Subscription with the appropriate price_id (configured per tier in environment variables).
- Webhook handler processes: checkout.session.completed (activate subscription), customer.subscription.updated (tier change), customer.subscription.deleted (downgrade to free / suspend), invoice.payment_failed (notify user).
- Quota is updated atomically in SubscriptionProfile on every webhook event to avoid race conditions.

5. AI Agent & RAG Architecture

5.1 Agent Design

The advisory agent is a multi-turn conversational AI with a structured discovery protocol. It is NOT a free-form chatbot — it follows a defined conversation graph while remaining natural and flexible.

Conversation Phases

- Phase 1 — GREETING: Brief welcome, set expectations (2–3 questions to understand their needs).
- Phase 2 — DISCOVERY: Ask about document types, languages, complexity (equations, tables, handwriting?), volume (docs/month), and integration requirements (API vs UI).
- Phase 3 — CLARIFICATION: If answers are ambiguous, ask 1 follow-up question maximum.
- Phase 4 — RETRIEVAL: Silently run RAG query based on summarised user profile.
- Phase 5 — RECOMMENDATION: Present 1 primary + 1 alternative tier with plain-language reasoning. Embed structured JSON payload in the response for frontend rendering.
- Phase 6 — DEMO HANDOFF: Invite the user to upload a sample document to see the result live.

5.2 System Prompt Design

The LLM system prompt has four injected sections assembled per-request:

- Static instruction block: defines the agent's persona, prohibited disclosures (no model names, no pricing figures — defer to the recommendation card), and conversation phase logic.
- Tier catalogue block: a compact JSON object listing tier slugs, public names, and their capability tags (e.g. `supports_equations: true`, `supports_handwriting: false`). This is injected as ground truth; the agent must ground recommendations in this catalogue.
- RAG context block: the top-K retrieved chunks from Pinecone. The agent is instructed to cite retrieved evidence when explaining its recommendation.
- Conversation history block: last N turns of the chat (N=10 by default, configurable by tier to manage token costs).

5.3 Recommendation JSON Schema

When the agent concludes its recommendation, it emits a structured payload within its response. The frontend detects a special delimiter and parses this into the recommendation card:

Recommendation Payload (embedded in agent response)

```
{
  "recommendation": {
    "primary_tier": "pro",
    "alternative_tier": "basic",
    "primary_reasons": ["Handles multi-column PDFs", "Equation extraction support"],
    "alternative_reasons": ["Lower cost if volume < 500 docs/month"],
    "demo_model_key": "pro"
  }
}
```

5.4 RAG Knowledge Base Structure

Two document types are ingested into Pinecone with distinct metadata:

Document Type	Structure & Metadata
Research Papers (PDF)	Chunked by paragraph. Metadata: source_type=research_paper, title, authors, year, capability_tags[] (e.g. ['equation_ocr', 'layout_analysis'])
Tier Spec Sheets (Markdown)	Chunked by section. Metadata: source_type=tier_spec, tier_id, tier_name, capability_tags[]

This dual-source design allows the agent to answer both 'why does this work technically?' (research papers) and 'what does this tier actually do?' (spec sheets) — giving richer, more credible recommendations.

6. OCR Processing Layer

6.1 Model Hosting Strategy

OCR models are sourced from HuggingFace Hub and hosted on cloud GPU instances. Two hosting options are supported and can be mixed per tier:

Hosting Mode	Description
HuggingFace Inference Endpoints	Managed, auto-scaling GPU endpoints provisioned directly from HuggingFace Hub. No infrastructure management needed. Best for small scale (< 1k users). Pay-per-request or dedicated instance.
Self-hosted on cloud VM	A GPU VM (e.g. AWS EC2 g4dn.xlarge or GCP T4) running a FastAPI inference server. Lower per-request cost at scale. Required if fine-tuned or private models are used.

At the current scale target (< 1k users), HuggingFace Inference Endpoints are recommended for all tiers as they eliminate DevOps overhead.

6.2 Recommended HuggingFace Models by Use Case

Tier / Use Case	Suggested HF Model	Notes
Basic — plain text PDFs	microsoft/trocr-base-printed	Fast, lightweight, high accuracy on clean printed text
Standard — mixed layouts	naver-clova-ix/donut-base	Document understanding, handles tables and form fields
Pro — scientific/equations	facebook/nougat-base	Academic PDF specialist; LaTeX equation output
Pro — handwriting	microsoft/trocr-large-handwritten	Trained on handwritten document datasets
Enterprise — complex docs	google/pix2struct-large	Scene-text + diagram understanding; highest accuracy

6.3 Async Job Processing with Celery

OCR inference is never run synchronously in the Django request cycle. All jobs go through Celery:

- Job submission: POST /ocr/jobs/ creates an OCRJob record with status=queued, uploads the file to S3, and enqueues a Celery task.

- Worker scaling: Celery workers run on a separate container/VM pool. Auto-scaling based on Redis queue depth (configured via Celery autoscale or Kubernetes HPA).
- Result polling: Frontend polls GET /ocr/jobs/{id}/ every 2 seconds, or subscribes to a Redis pub/sub channel for instant notification (SSE endpoint available).
- Retry policy: Failed jobs retry up to 3 times with exponential backoff. After max retries, status is set to failed and the user is notified.
- Demo jobs: Routed to a dedicated Celery queue (demo_queue) with a separate pool of workers running only the fast baseline model, to isolate demo latency from production jobs.

7. Database Schema (PostgreSQL)

7.1 Core Tables

Table	Key Columns	Notes
users	id, email, password_hash, is_active, created_at	Custom user model; email is unique identifier
subscription_profiles	user_id, stripe_customer_id, stripe_subscription_id, tier_id, quota_limit, quota_used, period_reset_at	1:1 with users; updated on every Stripe webhook
tiers	id, slug, public_name, quota_limit, stripe_price_id, is_active	Admin-managed; not exposed publicly by name except here
chat_sessions	id (UUID), user_id (nullable), created_at, last_activity_at, recommendation_tier_id	user_id is null for anonymous advisor sessions
chat_messages	id, session_id, role, content, metadata (JSON), created_at	metadata stores recommendation JSON payload when present
ocr_jobs	id (UUID), user_id, tier_id, input_s3_key, result_s3_key, status, error_message, created_at, completed_at	Main job tracking table
knowledge_documents	id, title, doc_type, s3_key, pinecone_namespace, ingested_at, capability_tags (JSON)	Tracks ingested RAG documents
api_keys	id, user_id, key_hash, label, created_at, last_used_at, is_active	Hashed API keys for programmatic OCR access

8. Infrastructure & Deployment

8.1 Infrastructure Components

Component	Technology / Service
Frontend Hosting	Vercel (or AWS Amplify) — CDN-edge deployment, automatic preview URLs per PR
Backend Hosting	AWS ECS Fargate (containers) or a single EC2 instance for small scale
PostgreSQL	AWS RDS PostgreSQL 15 (db.t3.medium) — automated backups, Multi-AZ optional
Redis	AWS ElastiCache Redis 7 (cache.t3.micro) — Celery broker + rate limit store
File Storage	AWS S3 — input documents, OCR results, RAG source documents
OCR Inference	HuggingFace Inference Endpoints (EU or US region matching backend)
Vector Database	Pinecone Starter (serverless) — sufficient for < 1M vectors
LLM API	Anthropic Claude API (claude-sonnet-4) or OpenAI GPT-4o — configurable
Email	AWS SES — transactional emails (verification, receipts, usage alerts)
Payments	Stripe — subscriptions, invoicing, customer portal
Monitoring	Sentry (errors) + AWS CloudWatch (infra) + Flower (Celery dashboard)
CI/CD	GitHub Actions — test, lint, build Docker images, deploy on merge to main

8.2 Docker & Container Strategy

The backend is containerised. Four containers run in production:

- `django-web`: Gunicorn serving the synchronous DRF API (scales to N replicas).
- `django-asgi`: Uvicorn serving ASGI (SSE streaming endpoints for chat and job results).
- `celery-worker`: Celery workers processing OCR and RAG ingestion tasks.
- `celery-beat`: Celery Beat scheduler for periodic tasks (quota resets, usage reports).

All four images are built from the same Dockerfile with different CMD overrides, ensuring dependency parity.

8.3 Environment Variables (Key)

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string (Celery + cache)
AWS_ACCESS_KEY_ID / SECRET	S3 and SES credentials
S3_BUCKET_NAME	Bucket for uploads and results
PINECONE_API_KEY / ENVIRONMENT	Vector DB access
ANTHROPIC_API_KEY	LLM inference (or OPENAI_API_KEY)
STRIPE_SECRET_KEY / WEBHOOK_SECRET	Payment processing
HF_ENDPOINT_URL_{TIER_SLUG}	Per-tier HuggingFace endpoint URLs
DJANGO_SECRET_KEY	Django cryptographic signing
ALLOWED_HOSTS / CORS_ORIGINS	Security config
JWT_ACCESS_TOKEN_LIFETIME	Token expiry (default: 15 min)

9. Security Architecture

9.1 Authentication & Authorisation

- JWT tokens: short-lived access tokens (15 min) + longer-lived refresh tokens (7 days, rotated on use).
- OAuth via django-allauth: Google and GitHub; email from OAuth is verified by the provider.
- Tier-based permissions: DjangoModelPermissions extended with a custom HasActiveSubscription permission class that checks SubscriptionProfile.tier_id against the required tier for each endpoint.
- API key auth: Programmatic access uses hashed API keys (SHA-256). Keys are shown once on creation and never stored in plaintext.

9.2 Rate Limiting

- Anonymous advisor + demo endpoints: 10 requests/minute per IP (Redis-backed, sliding window).
- Demo file upload: additional limit of 5 uploads/hour per IP to prevent abuse of free inference.
- Authenticated API: per-tier quota enforced at the Django layer (checked pre-task, decremented on job completion).

9.3 Data Privacy

- Demo uploads: files stored in a dedicated S3 prefix with a 24-hour lifecycle policy — automatically deleted.
- Authenticated uploads: stored for 30 days by default; user can delete from dashboard.
- Chat sessions: anonymous sessions are purged after 48 hours of inactivity.
- No OCR content is used for model training without explicit user consent (toggle in account settings).

9.4 Infrastructure Security

- All traffic over HTTPS/TLS 1.3. HTTP requests are permanently redirected.
- Database and Redis are in private subnets, not internet-accessible.
- S3 bucket is private; all file access via pre-signed URLs with 1-hour expiry.
- Stripe webhook signature verification on every incoming event.
- Content Security Policy headers on all frontend responses.

10. Tier & Pricing Architecture

Tier names and model identities are kept separate by design. The database stores tier slugs internally; the frontend only renders `public_name` strings fetched from the API. This allows renaming tiers or swapping underlying models without any frontend code changes.

10.1 Recommended Tier Structure (Starting Point)

Tier Slug	Suggested Public Name	Capability Profile
free	Starter	PDF text extraction only; 50 pages/month; no API access
basic	Essential	Printed text + tables; 500 pages/month; API access included
pro	Professional	All of Essential + equations, multi-language, handwriting; 5,000 pages/month
enterprise	Enterprise	All capabilities + custom model fine-tuning option; unlimited; dedicated support

Note: These are suggested defaults. Tier slugs, names, capabilities, quotas, and Stripe price IDs are all admin-configurable via Django Admin without code deployments.

10.2 Stripe Integration Flow

- Each tier maps to a Stripe Product with a recurring monthly Price (and optionally an annual Price).
- `POST /billing/checkout/` creates a Stripe Checkout Session in subscription mode, pre-populating the customer email and selected tier's `price_id`.
- On `checkout.session.completed` webhook: activate tier, set quota limits, send welcome email.
- Stripe Customer Portal (`GET /billing/portal/`) allows users to upgrade, downgrade, or cancel without any custom UI needed.

11. Development Roadmap

Phase 1 — Foundation (Weeks 1–4)

- Django project setup: custom User model, JWT auth, DRF base, PostgreSQL config.
- Basic tier model + Stripe products created.
- S3 bucket + HuggingFace endpoint for one baseline OCR model.
- Simple demo upload endpoint (no auth, one model, basic result return).
- Next.js project scaffolded: landing page skeleton + basic demo upload UI.

Phase 2 — AI Advisor (Weeks 5–8)

- RAG ingestion pipeline: PDF/Markdown upload → chunking → embedding → Pinecone upsert.
- LangChain RAG chain + system prompt template wired to the LLM API.
- Chat session models + POST /advisor/message/ with SSE streaming.
- Frontend: full chat UI with streaming text rendering + recommendation card component.
- Tier catalogue seeded in DB and spec sheets ingested into Pinecone.

Phase 3 — Checkout & Dashboard (Weeks 9–11)

- Stripe checkout flow + webhook handler fully implemented.
- Authenticated OCR job submission + Celery async processing.
- User dashboard: API key management, usage meters, job history.
- All tier-specific OCR models integrated and mapped.

Phase 4 — Polish & Launch (Weeks 12–14)

- Landing page design finalised (animations, pricing section, testimonials).
- Security hardening: rate limits, CSP headers, pen-test of auth flows.
- Monitoring + alerting configured (Sentry, CloudWatch alarms).
- Admin tooling: Django Admin views for tier management and knowledge base uploads.
- Soft launch to beta users → gather feedback → iterate.

12. Architecture Decision Log

Decision	Choice Made	Rationale
LLM provider	Anthropic Claude (configurable)	Best instruction-following for structured discovery conversations; model is swappable via env var
Vector DB	Pinecone (serverless)	Managed, no-ops, free tier sufficient at small scale; easy to migrate to self-hosted Weaviate later
OCR hosting	HuggingFace Endpoints	Eliminates GPU infra management; cost-effective at < 1k users; swap to self-hosted when unit economics improve
Async processing	Celery + Redis	Django-native, well-documented, simple ops; overkill could be avoided with threading at very small scale but Celery allows future scaling
Payments	Stripe Subscriptions	Stripe's customer portal eliminates need to build upgrade/downgrade UI; webhook-driven model is reliable
Frontend framework	Next.js	SSR for SEO on landing page, React for interactive chat/demo, single codebase
Model names hidden	Tier slugs in DB only	Preserves ability to swap models per tier without customer-facing changes; protects IP
Demo = unauthenticated	Yes, rate-limited	Removes friction for conversion; rate limiting prevents abuse; demo always uses fast model to minimise cost

Document End — Architecture v1.0

This document covers the full system design for the Unified OCR Platform.

All technology choices, data models, and API contracts are subject to revision during Phase 1.

Next step: review with engineering team, confirm tier count and Stripe product creation.